

CS2220: Introduction to Computational Biology

Finding Regulatory Motifs in DNA Sequences



Somayyeh Koochi

Fall 2024



Adapted with modifications from lecture notes prepared by Phillip Compeau,

Bioinformatics Algorithms: An Active Learning Approach

Outline



- ❧ **What is Motif**
- ❧ Motif finding problem
- ❧ Brute Force Motif Finding
- ❧ The Median String Problem
- ❧ Search Trees
- ❧ Branch-and-Bound Motif Search
- ❧ Branch-and-Bound Median String Search

What is a DNA Motif?



- ❖ DNA motifs are short, recurring patterns that are presumed to have a biological function.
 - As regulatory motifs may vary at some positions
- Every plant cell keeps track of day and night independently of other cells
 - e.g. three plant genes, called *LCY*, *CCA1*, and *TOC1*, are the clock's master timekeepers
 - Such regulatory genes, and the regulatory proteins that they encode, are often controlled by external factors (e.g., nutrient availability or sunlight) in order to allow organisms to adjust their gene expression

Regulatory Regions

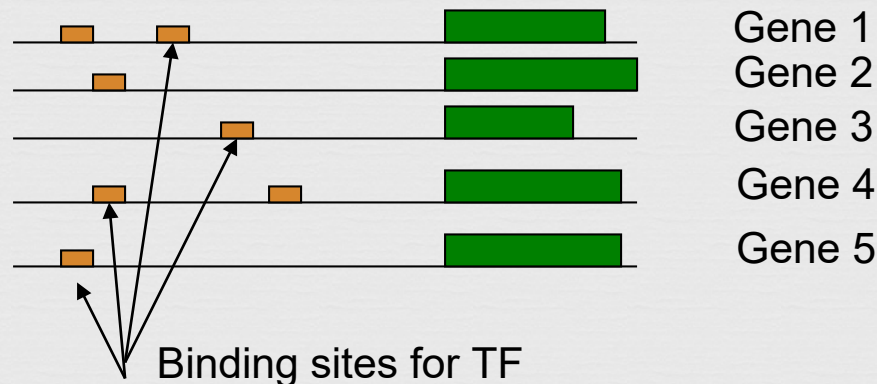


- ✧ Every gene contains a regulatory region (RR) typically stretching 100-1000 bp upstream of the transcriptional start site
- ✧ Located within the RR are the *Transcription Factor Binding Sites* (TFBS), also known as *motifs*, specific for a given transcription factor
- ✧ A TFBS can be located anywhere within the Regulatory Region.
- ✧ TFBS may vary slightly across different regulatory regions
- ✧ Regulatory protein (TF) binds to TFBS
- ✧ So finding the same motif in multiple genes' regulatory regions suggests a regulatory relationship amongst those genes

Motif in Regulatory Regions



- Suppose a transcription factor (TF) controls five different genes
- Each of the five genes should have binding sites for TF in their promoter region



Outline



- ❧ What is Motif
- ❧ **Motif finding problem**
- ❧ Brute Force Motif Finding
- ❧ The Median String Problem
- ❧ Search Trees
- ❧ Branch-and-Bound Motif Search
- ❧ Branch-and-Bound Median String Search

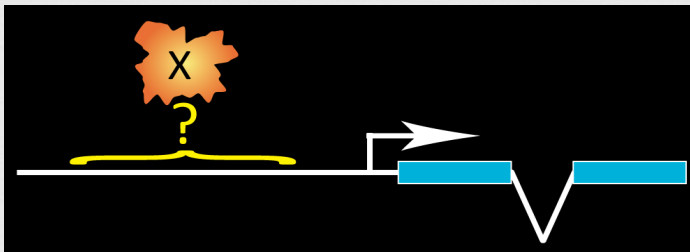
Two Methods



Pattern Matching

Finding known motifs

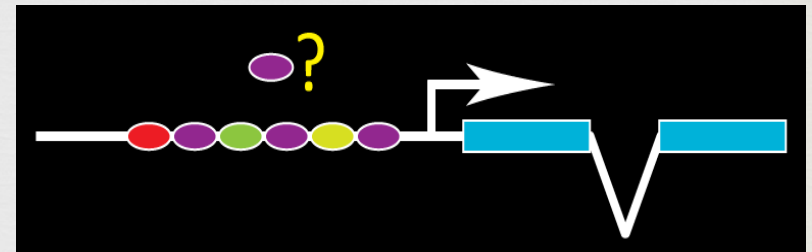
- Does protein X bind upstream of my genes?



Pattern Discovery

Finding unknown motifs

- What motifs are upstream of my genes?



Motif Finding Problem



- ❧ Now suppose we are given the promoter regions of the five genes G1, G2, ... G5
- ❧ Can we find the binding sites of TF, without knowing about them *a priori* ?
 - ❧ Binding sites are similar to each other, but not necessarily identical
- ❧ This is the motif finding problem
- ❧ To find a motif that represents binding sites of an unknown TF

Identifying Motifs: Complications



- ❧ We do not know the motif sequence
 - ❧ May know its length
- ❧ We do not know where it is located relative to the genes start
- ❧ Motifs can differ slightly from one gene to the next
 - ❧ Non-essential bases could mutate...

HOW TO DISCERN FUNCTIONAL MOTIFS FROM RANDOM ONES?

Sample Problem



- Find a motif in a sample of
- 20 “random” sequences (e.g. 600 nt long)
 - each sequence containing an implanted pattern of length 15,
 - each pattern appearing with 4 mismatches as (15,4)-motif.

The Motif Finding Problem



Given a random sample of DNA sequences:

```
cctgatagacgctatctggctatccacgtacgtaggtcctctgtgCGAATctatgCGTTTTccaaccat
agtactggtgtacatttgatacgtacgtacaccggcaacctgaaacaaacgctcagaaccagaagtgc
aaacgtacgtgcaccctctttcttcgtggctctggccaacgagggctgatgtataagacgaaaatttt
agcctccgatgtaagtcatacgtgtaactattacctgccacccctattacatcttacgtacgtataca
ctgttatacaacgcgctcatggcggggtatgCGTTTTggtcgctcgtagctcgatcgTTAACgtacgtc
```

Find the pattern that is implanted in each of the individual sequences, namely, the motif

The Motif Finding Problem

(cont'd)



Additional information:

- ✧ The hidden sequence is of length 8
- ✧ The pattern is not exactly the same in each array because random point mutations may occur in the sequences

The Motif Finding Problem

(cont'd)



✧ The patterns revealed with no mutations:

```

cctgatatagacgctatctggctatccacgtacgtaggctcctctgtgCGaatctatgcgtttccaaccat
agtactggtgtacatttgaacgtacgtacacccggcaacctgaaacaaacgctcagaaccagaagtgc
aaacgtacgtcaccctctttcttctgtggctctggccaacgagggctgatgtataagacgaaaatttt
agcctccgatgtaagtcataacgtacgtgtaactattacctgccaccctattacatcttacgtacgtataca
ctgtttatacaacgcgctcatggcgggggtatgcgttttggctcgtcgtacgctcgatcgttacgtacgtc
  
```

acgtacgt

Consensus String

The Motif Finding Problem

(cont'd)



✧ The patterns with 2 point mutations:

cctgatatagacgctatctggctatccaGgtacTtaggtcctctgtgCGaatctatgcgttttccaaccat
agtactgggtgtacattttgatCcAtacgtacaccggcaacctgaaacaaacgctcagaaccagaagtgc
aaacgtTAgtgcaccctctttcttcgtggctctggccaacgagggctgatgtataagacgaaaatttt
agcctccgatgtaagtcatactgtaactattacctgccacccctattacatcttacgtCcAtataca
ctgttatacaacgcgtcatggcggggtatgcgtttttggtcgtcgtacgctcgatcgттаCcgtacgGc

The Motif Finding Problem

(cont'd)



✧ The patterns with 2 point mutations:

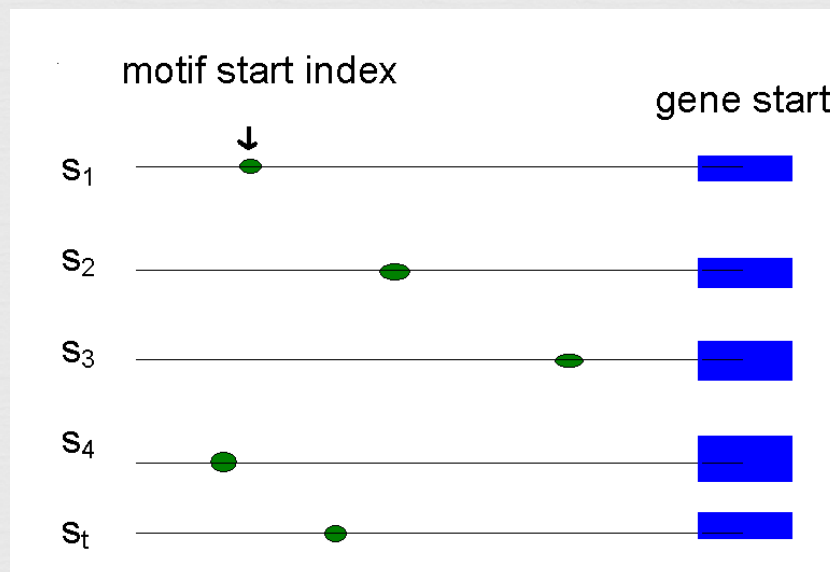
cctgatagacgctatctggctatccaGgtacTtaggtcctctgtgCGaatctatgcgttttccaaccat
agtactgggtgtacatttgatCcAtacgtacaccggcaacctgaaacaaacgctcagaaccagaagtgc
aaacgtTAgtgcaccctctttcttcgtggctctggccaacgagggctgatgtataagacgaaaatttt
agcctccgatgtaagtcatagctgtaactattacctgccacccctattacatcttacgtCcAtataca
ctgttatacaacgcgtcatggcggggtatgcgtttttggtcgtcgtacgctcgatcgттаCcgtacgGc

CAN WE STILL FIND THE MOTIF?
(NOW THAT WE HAVE 2 MUTATIONS)

Defining Motifs



- ✧ To define a motif, let's say we know where the motif starts in the sequence
- ✧ The motif start positions in their sequences can be represented as $s = (s_1, s_2, s_3, \dots, s_t)$



Motifs: Profiles and Consensus



Alignment

a	G	g	t	a	c	T	t
C	c	A	t	a	c	g	t
a	c	g	t	T	A	g	t
a	c	g	t	C	c	A	t
C	c	g	t	a	c	g	G

↻ Line up the patterns by their start indexes

$$\mathbf{s} = (s_1, s_2, \dots, s_t)$$

Profile

A	3	0	1	0	3	1	1	0
C	2	4	0	0	1	4	0	0
G	0	1	4	0	0	0	3	1
T	0	0	0	5	1	0	1	4

↻ Construct matrix profile with frequencies of each nucleotide in columns

Consensus A C G T A C G T

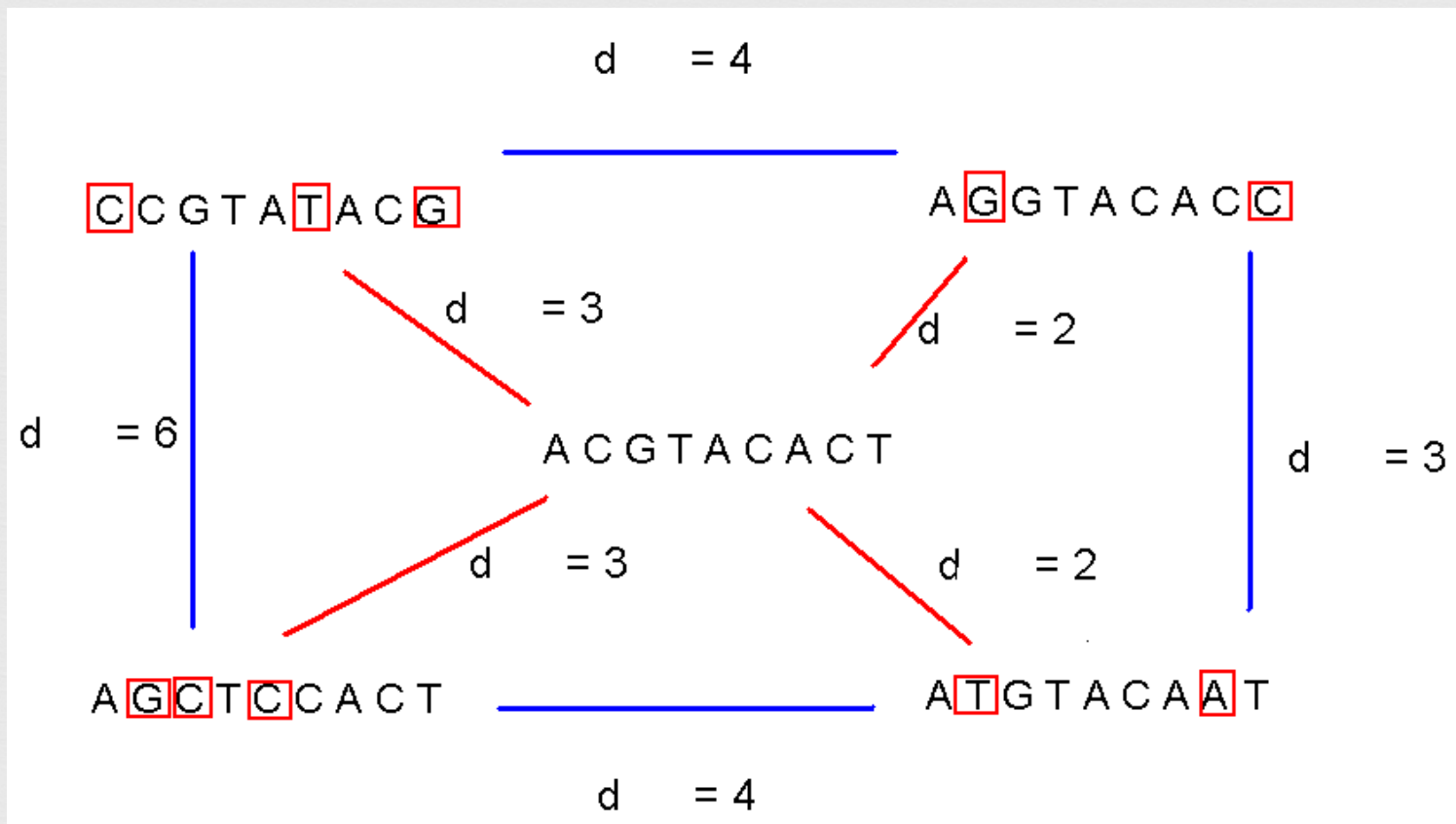
↻ Consensus nucleotide in each position has the highest score in column

Consensus



- ❧ Think of consensus as an “**ancestor**” motif, from which mutated motifs emerged
- ❧ The *distance* between a real motif and the consensus sequence is generally less than that for two real motifs

Consensus (cont'd)



Evaluating Motifs



- ✧ We have a guess about the consensus sequence, but
HOW “GOOD” IS THIS CONSENSUS?
- ✧ Need to introduce a scoring function to compare different guesses and choose the “best” one.

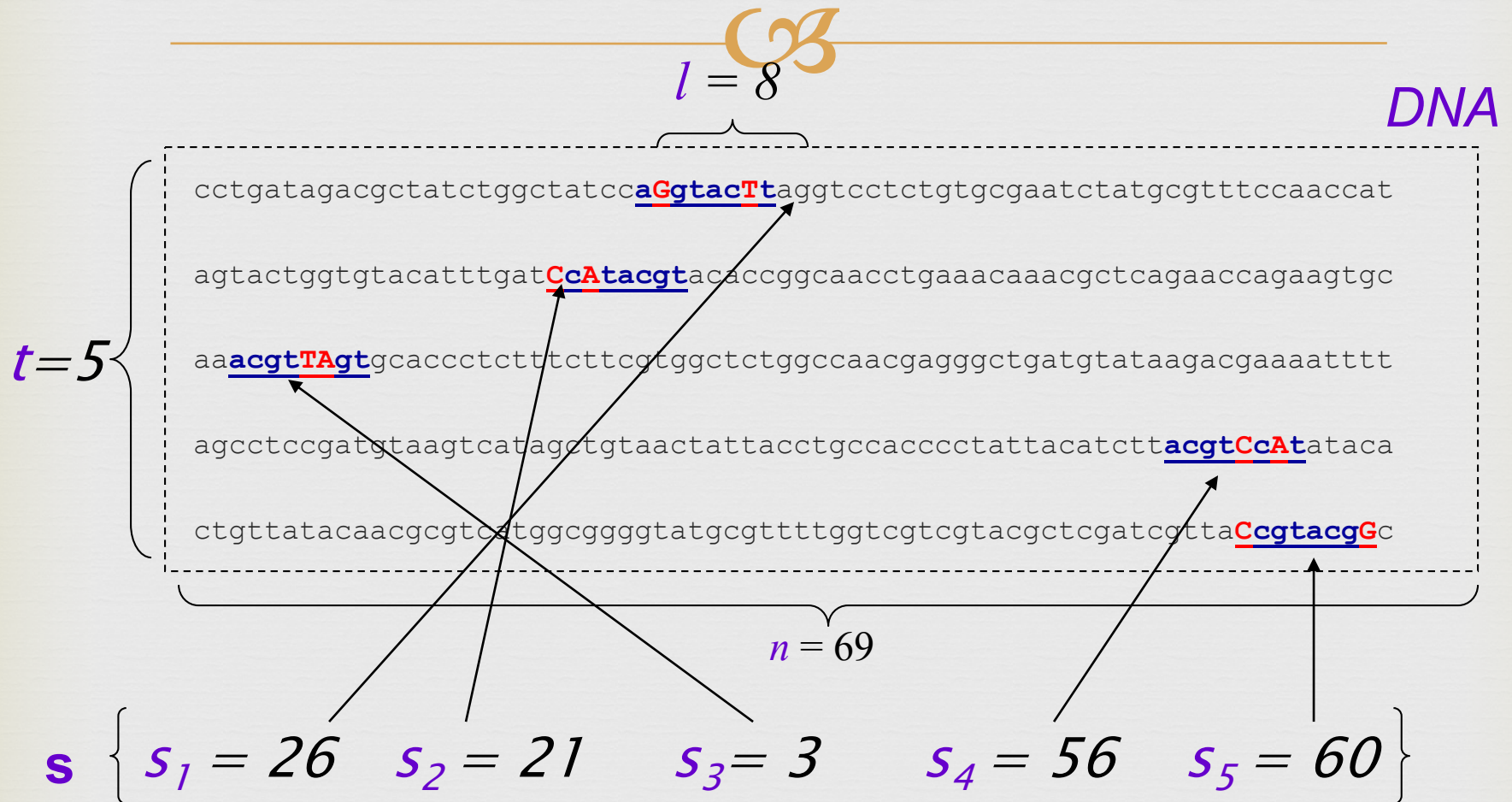
Defining Some Terms



- ✧ t - number of sample DNA sequences
- ✧ n - length of each DNA sequence
- ✧ DNA - sample of DNA sequences ($t \times n$ array)

- ✧ l - length of the motif (l -mer)
- ✧ s_i - starting position of an l -mer in sequence i
- ✧ $\mathbf{s} = (s_1, s_2, \dots, s_t)$ - array of motif's starting positions

Parameters



Scoring Motifs



Given $\mathbf{s} = (s_1, \dots, s_t)$ and **DNA**:

$$\text{Score}(\mathbf{s}, \text{DNA}) = \sum_{i=1}^l \max_{k \in \{A, T, C, G\}} \text{count}(k, i)$$

l							
a	G	g	t	a	c	T	t
C	c	A	t	a	c	g	t
a	c	g	t	T	A	g	t
a	c	g	t	C	c	A	t
C	c	g	t	a	c	g	G
t							

A	3	0	1	0	3	1	1	0
C	2	4	0	0	1	4	0	0
G	0	1	4	0	0	0	3	1
T	0	0	0	5	1	0	1	4

Consensus **a c g t a c g t**

Score **3+4+4+5+3+4+3+4=30**

The Motif Finding Problem



✧ If starting positions $\mathbf{s}=(s_1, s_2, \dots s_t)$ are given, finding consensus is easy even with mutations in the sequences because we can simply construct the profile to find the motif (consensus)

✧ But... the starting positions $\mathbf{s}$ are usually not given.

HOW CAN WE FIND THE “BEST” PROFILE MATRIX?

The Motif Finding Problem: Formulation



- ❧ Goal: Given a set of DNA sequences, find a set of ℓ -mers, one from each sequence, that maximizes the consensus score
- ❧ Input: A $t \times n$ matrix of *DNA*, and ℓ , the length of the pattern to find
- ❧ Output: An array of t starting positions $\mathbf{s} = (s_1, s_2, \dots, s_t)$ maximizing $\text{Score}(\mathbf{s}, \text{DNA})$

Information Theory



- Information theory is a branch of applied mathematics involved with the quantification of information
- It has been applied to DNA motifs in order to determine the amount of uncertainty at each position in a site

$$H(p_1, \dots, p_N) = - \sum_{i=1}^N p_i \cdot \log_2(p_i).$$

- Information is a decrease in uncertainty
 - 1 base occurs every time - 2 bits
 - 2 bases occur 50% of time - 1bit
 - 4 bases occur equally - 0 bits

Motif Logo



- ✧ The five motifs in five different genes have mutations in position 3 and 5
- ✧ Representations called *motif logos* illustrate the conserved and variable regions of a motif

TGGGGGA
TGAGAGA
TGGGGGA
TGAGAGA
TGAGGGA



The Matrix



- A position weight matrix (PWM)
 - also called position-specific weight matrix (PSWM)
 - also called position-frequency matrix (PFM)
 - also called position-specific scoring matrix (PSSM)
 - or just matrix
- There is a matrix element for all possible bases at every position.

	1	2	3	4	5	6	7	8	9	10	11
A	4	13	5	3	0	0	0	0	17	0	6
C	4	1	2	0	0	0	0	0	0	1	0
G	3	3	0	0	18	0	0	0	1	4	3
T	7	1	11	15	0	18	18	18	0	13	9

The Matrix



Counts

A	4	13	5	3	0	0	0	0	17	0	6
C	4	1	2	0	0	0	0	0	0	1	0
G	3	3	0	0	18	0	0	0	1	4	3
T	7	1	11	15	0	18	18	18	0	13	9

Frequency

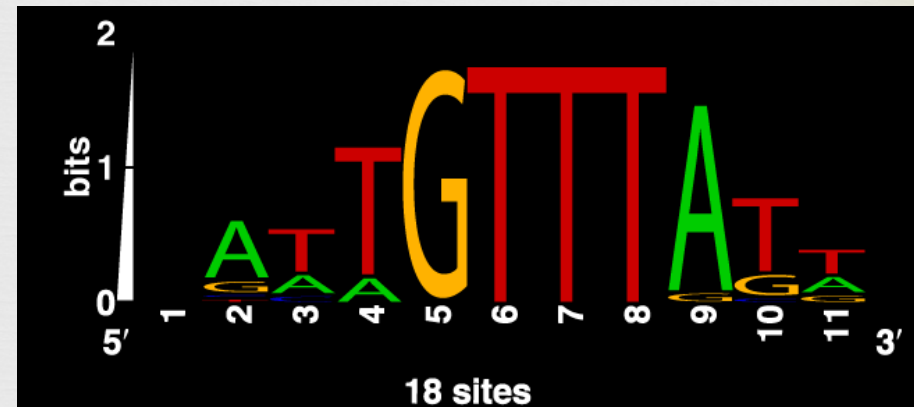
A	0.2	0.7	0.3	0.2	0.0	0.0	0.0	0.0	0.9	0.0	0.3
C	0.2	0.1	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.0
G	0.2	0.2	0.0	0.0	1.0	0.0	0.0	0.0	0.1	0.2	0.2
T	0.4	0.1	0.6	0.8	0.0	1.0	1.0	1.0	0.0	0.7	0.5

Sequence Logos



- A visual representation of the sequence
- Each column of the matrix is represented as a stack of letters whose size is proportional to the corresponding residue frequency
- The total height of each column is two minus its information content.

A	4	13	5	3	0	0	0	0	17	0	6
C	4	1	2	0	0	0	0	0	0	1	0
G	3	3	0	0	18	0	0	0	1	4	3
T	7	1	11	15	0	18	18	18	0	13	9



Motifs

T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C

SCORE(*Motifs*) 3 + 4 + 0 + 0 + 1 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30

COUNT(*Motifs*)

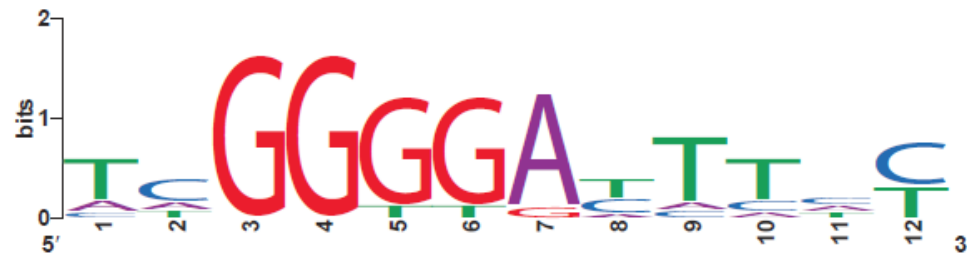
A:	2	2	0	0	0	0	9	1	1	1	3	0
C:	1	6	0	0	0	0	0	4	1	2	4	6
G:	0	0	10	10	9	9	1	0	0	0	0	0
T:	7	2	0	0	1	1	0	5	8	7	3	4

PROFILE(*Motifs*)

A:	.2	.2	0	0	0	0	.9	.1	.1	.1	.3	0
C:	.1	.6	0	0	0	0	0	.4	.1	.2	.4	.6
G:	0	0	1	1	.9	.9	.1	0	0	0	0	0
T:	.7	.2	0	0	.1	.1	0	.5	.8	.7	.3	.4

CONSENSUS(*Motifs*)

T C G G G G A T T T C C



Outline



- ❧ What is Motif
- ❧ Motif finding problem
- ❧ **Brute Force Motif Finding**
- ❧ The Median String Problem
- ❧ Search Trees
- ❧ Branch-and-Bound Motif Search
- ❧ Branch-and-Bound Median String Search

The Motif Finding Problem: Brute Force Solution



- ✧ Compute the scores for each possible combination of starting positions **s**
- ✧ The best score will determine the best profile and the consensus pattern in **DNA**
- ✧ The goal is to maximize $Score(\mathbf{s}, \mathbf{DNA})$ by varying the starting positions s_i , where:

$$s_i = [1, \dots, n - \ell + 1]$$
$$i = [1, \dots, t]$$

BruteForceMotifSearch



1. BruteForceMotifSearch(DNA, t, n, ℓ)
2. $bestScore \leftarrow 0$
3. for each $s = (s_1, s_2, \dots, s_t)$ from $(1, 1, \dots, 1)$
to $(n - \ell + 1, \dots, n - \ell + 1)$
4. if ($Score(s, DNA) > bestScore$)
5. $bestScore \leftarrow score(s, DNA)$
6. $bestMotif \leftarrow (s_1, s_2, \dots, s_t)$
7. return $bestMotif$

Running Time of BruteForceMotifSearch



- ✧ Varying $(n - \ell + 1)$ positions in each of t sequences, we're looking at $(n - \ell + 1)^t$ sets of starting positions
- ✧ For each set of starting positions, the scoring function makes ℓ operations, so complexity is $\ell(n - \ell + 1)^t = O(\ell n^t)$
- ✧ That means that for $t = 8$, $n = 1000$, $\ell = 10$ we must perform approximately 10^{20} computations !!!!!

ORDER OF THE ALGORITHM?

Outline



- ❧ What is Motif
- ❧ Motif finding problem
- ❧ Brute Force Motif Finding
- ❧ **The Median String Problem**
- ❧ Search Trees
- ❧ Branch-and-Bound Motif Search
- ❧ Branch-and-Bound Median String Search

The Median String Problem



- Given a set of t DNA sequences find a pattern that appears in all t sequences with the minimum number of mutations
- This pattern will be the motif

Hamming Distance



✧ Hamming distance:

✧ $d_H(v, w)$ is the number of nucleotide pairs that do not match when v and w are aligned. For example:

$$d_H(\text{AAAAAA}, \text{ACAAAC}) = 2$$

Total Distance: An Example



Given $v = \text{"acgtacgt"}$ and s

$$d_H(v, x) = 0$$

acgtacgt
acgtacgt

cctgatagacgctatctggctatcgaacgtacgttaggtcctctgtgcgaatctatgcggtttccaacat

$$d_H(v, x) = 0$$

acgtacgt
acgtacgt

agtactggtgtacatttgatgaacgtacgtacaccggcaacctgaaacaaacgctcagaaccagaagtgc

acgtacgt
acgtacgt

$$d_H(v, x) = 0$$

aaacgtacgtgcaccctctttcttcgtggctctggccaacgagggctgatgtataagacgaaaatttt

$$d_H(v, x) = 0$$

acgtacgt
acgtacgt

agcctccgatgtaagtcatactgtaactattacctgccaccctattacatcttgaacgtacgtataca

$$d_H(v, x) = 0$$

acgtacgt
acgtacgt

ctgttatacaacgcgtcatggcggggtatgcgttttggctcgtcgtacgctcgcgtacgtacgtc

v is the sequence in red, x is the sequence in blue

$TotalDistance(v, DNA) = 0$

Total Distance: An Example



Given $v = \text{"acgtacgt"}$ and s

$$d_H(v, x) = 1$$

acgtacgt

cctgatagacgctatctggctatccacgtacgtataggctcctctgtgccaatctatgcgtttccaacat

$$d_H(v, x) = 0$$

acgtacgt

agtactgggtgtacatttgatacgtacgtacaccggcaacctgaaacaaacgctcagaaccagaagtgc

acgtacgt

$$d_H(v, x) = 2$$

aaaacgtacgtgcaccctctttcttcgtggctctggccaacgagggctgatgtataagacgaaaatttt

$$d_H(v, x) = 0$$

acgtacgt

agcctccgatgtaagtcataagctgtaactattacctgccaccctattacatcttacgtacgtatataca

$$d_H(v, x) = 1$$

acgtacgt

ctgttatacaacgcgtcatggcggggtatgcgttttggtcgtcgctacgctcgatcggttaacgtacgtc

v is the sequence in red, x is the sequence in blue

$$\text{TotalDistance}(v, \text{DNA}) = 1 + 0 + 2 + 0 + 1 = 4$$

Total Distance: Definition



- Consider the motif v
- For each DNA sequence i , compute all $d_H(v, x)$, where x is an ℓ -mer with starting position s_i ($1 \leq s_i \leq n - \ell + 1$)
- $d_H(v, DNA_i) = \text{minimum of } d_H(v, x) \text{ among all starting position } s_i$
- $TotalDistance(v, DNA)$ is the sum of the minimum Hamming distances for all DNA sequences

The Median String Problem: Formulation



- ❧ Goal: Given a set of DNA sequences, find a median string
- ❧ Input: A $t \times n$ matrix DNA, and ℓ , the length of the pattern to find
- ❧ Output: A string v of ℓ nucleotides that **minimizes** $TotalDistance(v, DNA)$ over all strings of that length

Median String Search Algorithm



MedianStringSearch (DNA, t, n, ℓ)

1. $bestWord \leftarrow AAA...A$
2. $bestDistance \leftarrow \infty$
3. **for each ℓ -mer s from $AAA...A$ to $TTT...T$**
4. **if** $TotalDistance(s, DNA) < bestDistance$
5. $bestDistance \leftarrow TotalDistance(s, DNA)$
6. $bestWord \leftarrow s$
7. **return** $bestWord$

Motif Finding: BruteForceMotifSearch



Recall the BruteForceMotifSearch:

1. BruteForceMotifSearch(DNA, t, n, l)
2. $bestScore \leftarrow 0$
3. **for each** $s = (s_1, s_2, \dots, s_t)$ **from** $(1, 1, \dots, 1)$ **to** $(n-l+1, \dots, n-l+1)$
4. **if** $(Score(s, DNA) > bestScore)$
5. $bestScore \leftarrow Score(s, DNA)$
6. $bestMotif \leftarrow (s_1, s_2, \dots, s_t)$
7. **return** $bestMotif$

Motif Finding Problem vs. Median String Problem



✧ The *Motif Finding* is a **maximization** problem while *Median String* is a **minimization** problem

✧ However, the *Motif Finding* problem and *Median String* problem are computationally equivalent

ARE THEY DIFFERENT OR EQUIVALENT?

✧ Need to show that minimizing *TotalDistance* is equivalent to maximizing *Score*

We are looking for the same thing



		ℓ a G g t a c T t C c A t a c g t a c g t T A g t a c g t C c A t C c g t a c g G <							
--	--	---	--	--	--	--	--	--	--

At any column i
 $Score_i + TotalDistance_i = t$

Because there are ℓ columns
 $Score + TotalDistance = \ell * t$

Rearranging:
 $Score = \ell * t - TotalDistance$

$\ell * t$ is constant the minimization of the right side is equivalent to the maximization of the left side

Motif Finding Problem vs. Median String Problem



- ❧ Why bother reformulating the Motif Finding problem into the Median String problem?
- ❧ The Motif Finding Problem needs to examine all the combinations for s . That is $(n - \ell + 1)^t$ combinations!!! $\rightarrow$ runtime: $(n - \ell + 1)^t \ell t$
- ❧ The Median String Problem needs to examine all 4^ℓ combinations for v . This number is relatively smaller $\rightarrow$ runtime: $4^\ell n t$

Outline



- ❧ What is Motif
- ❧ Motif finding problem
- ❧ Brute Force Motif Finding
- ❧ The Median String Problem
- ❧ **Search Trees**
- ❧ Branch-and-Bound Motif Search
- ❧ Branch-and-Bound Median String Search

Structuring the Search



⌘ How can we perform the line

for each ℓ -mer s from AAA...A to TTT...T

⌘ We need a method for efficiently structuring and navigating the many possible motifs

⌘ This is not very different than exploring all t -digit numbers

Alternative Representation of the Search Space



- Let $A = 1, C = 2, G = 3, T = 4$
- Then the sequences from $AA...A$ to $TT...T$ become:

ℓ

$\{$

11...11

11...12

11...13

11...14

⋮

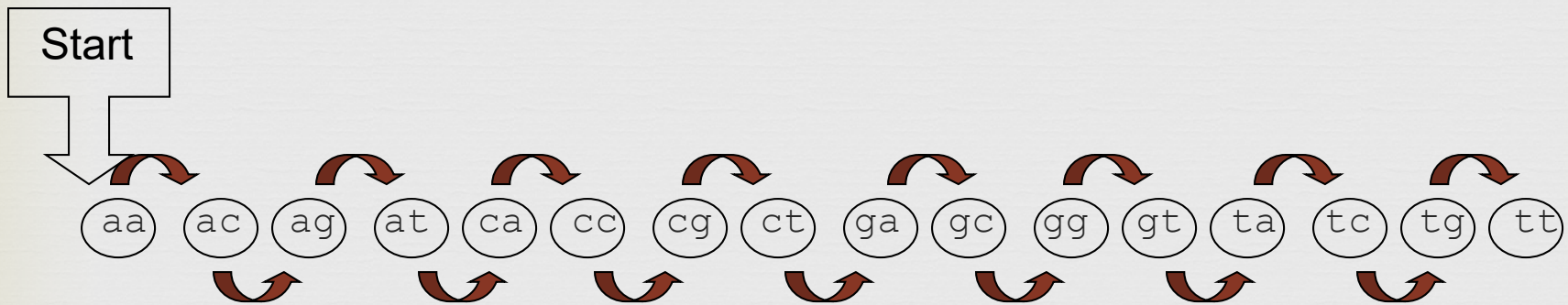
44...44

- Notice that the sequences above simply list all numbers as if we were counting on base 4 without using 0 as a digit

1. Linked List



Suppose $l = 2$

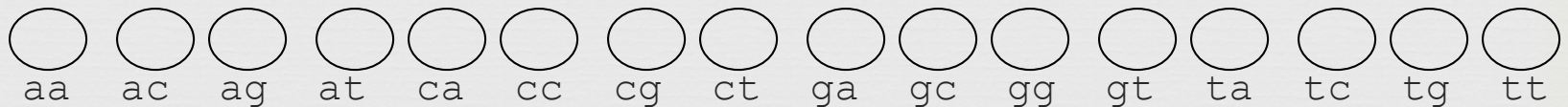


Need to visit all the predecessors of a sequence before visiting the sequence itself

Linked List (cont'd)



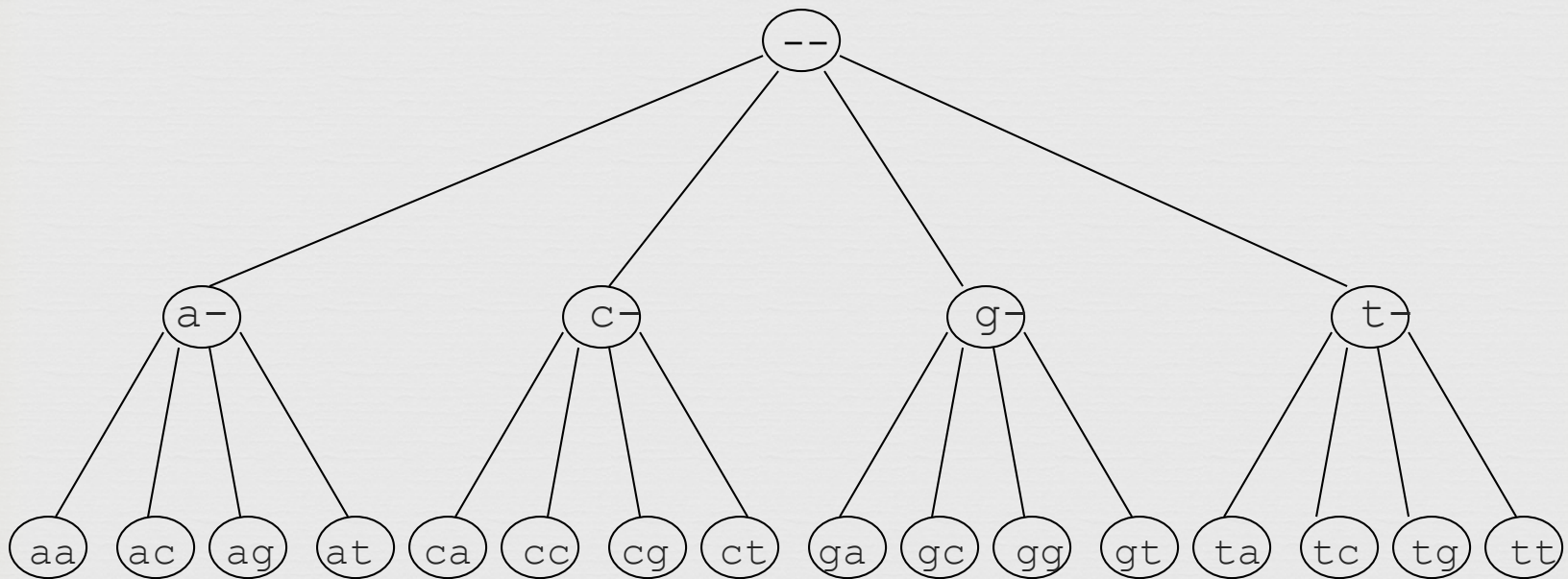
- Linked list is not the most efficient data structure for motif finding
- Let's try grouping the sequences by their prefixes



2. Search Tree



root



Analyzing Search Trees



- ❧ Characteristics of the search trees:
 - ❧ The sequences are contained in its leaves
 - ❧ The parent of a node is the prefix of its children
- ❧ How can we move through the tree?

Moving through the Search Trees



✧ Four common moves in a search tree that we are about to explore:

1. Visit the next leaf
2. Visit all the leaves
3. Visit the next vertex
4. Bypass the children of a node

1. Visit the Next Leaf



Given a current leaf **a** , we need to compute the “next” leaf:

1. NextLeaf(**a**, L , k) // **a** : the array of digits
2. **for** $i \leftarrow 1$ to L // L : length of the array
3. **if** $a_i < k$ // k : max digit value
4. $a_i \leftarrow a_i + 1$
5. **return** **a**
6. **else** $a_i \leftarrow 1$
7. **return** **a**

NextLeaf (cont'd)

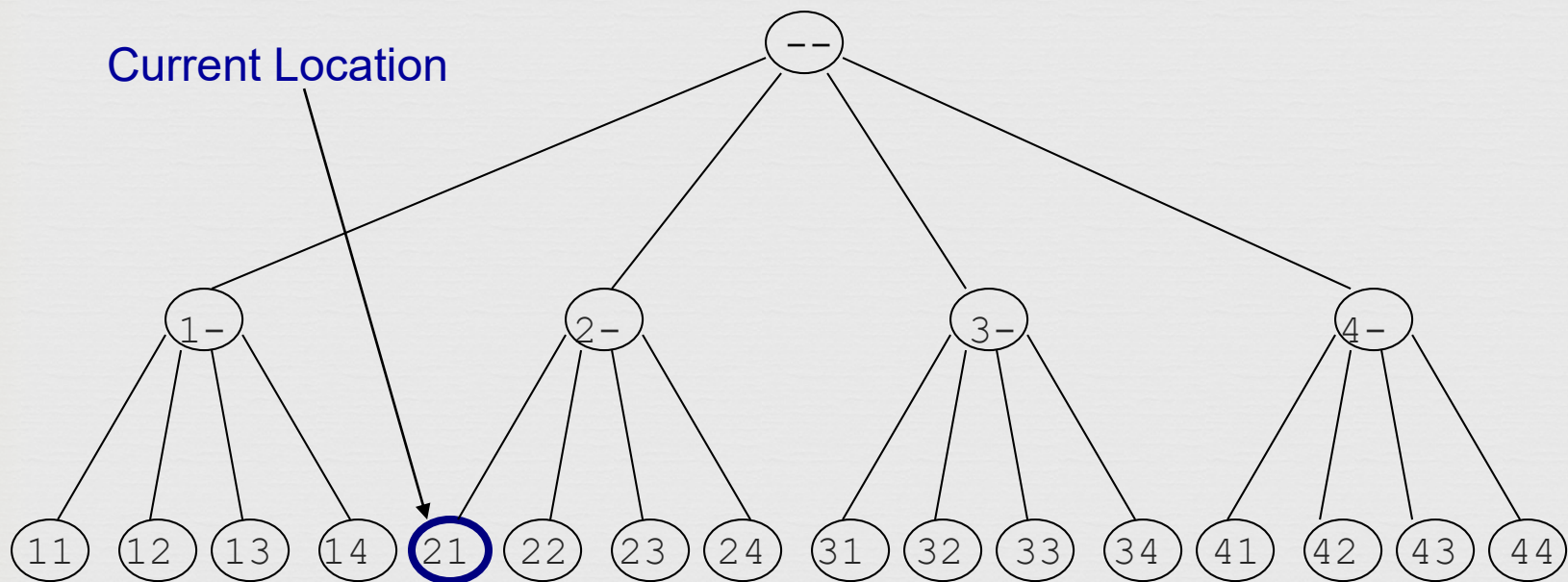


- ✧ The algorithm is common addition in radix k :
- ✧ Increment the least significant digit
- ✧ “Carry the one” to the next digit position when the digit is at maximal value

NextLeaf: Example



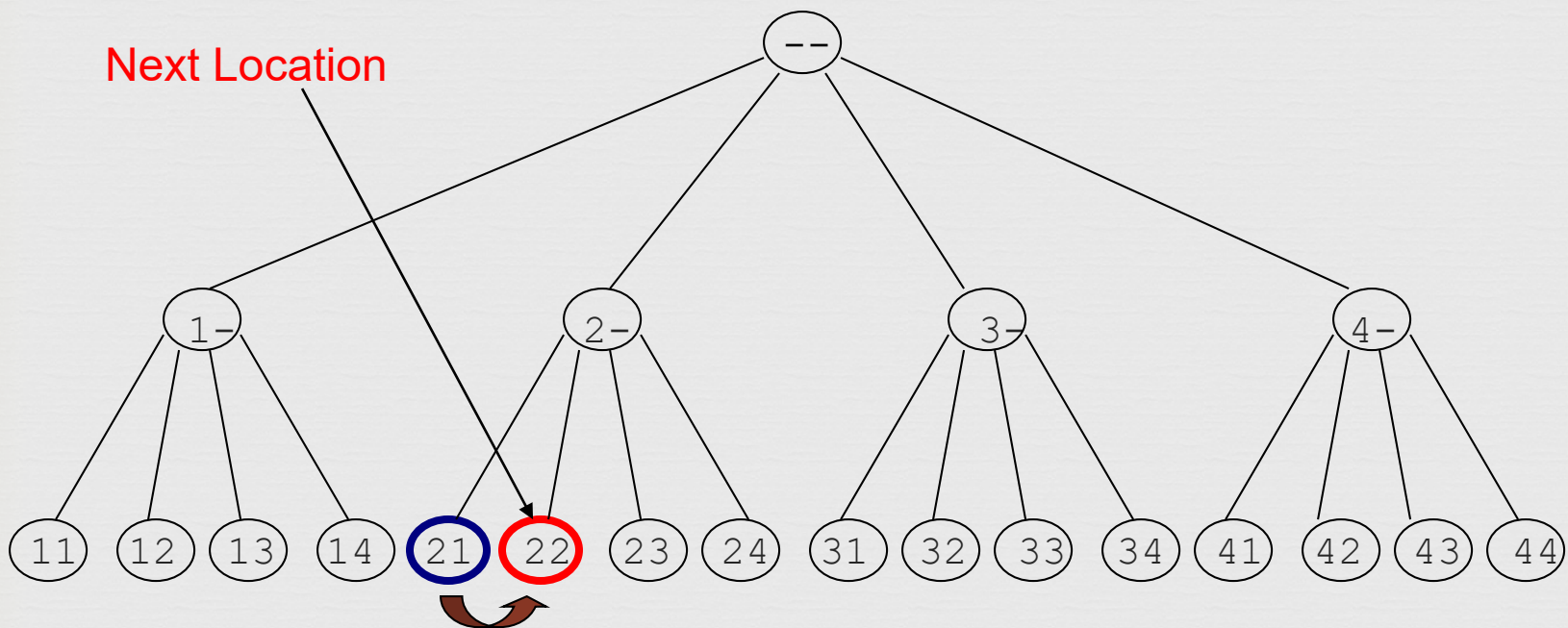

 Moving to the next leaf:



NextLeaf: Example (cont'd)




 Moving to the next leaf:



2. Visit All Leaves



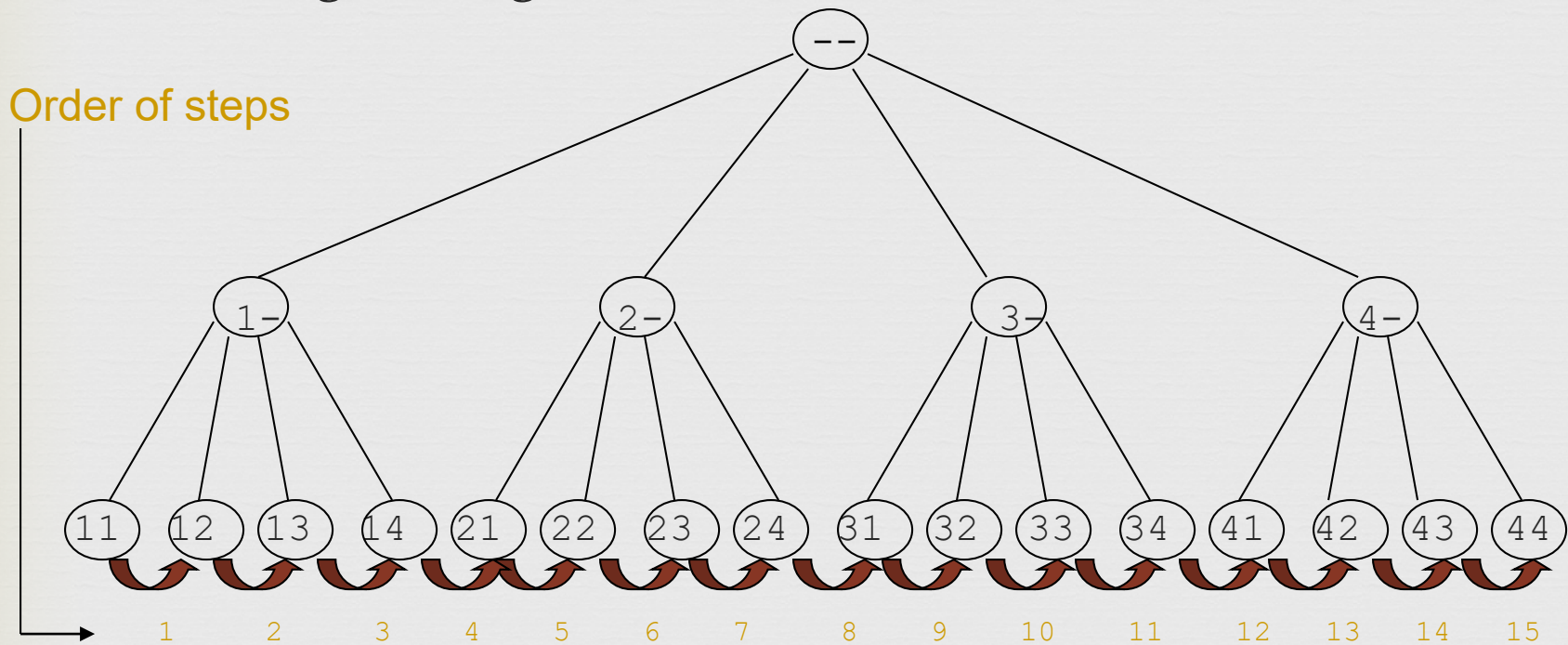
☞ Printing all permutations in ascending order:

1. AllLeaves(L, k) // L : length of the sequence
2. $\mathbf{a} \leftarrow (1, \dots, 1)$ // k : max digit value
3. **while** forever // $\mathbf{a}$: array of digits
4. output $\mathbf{a}$
5. $\mathbf{a} \leftarrow \text{NextLeaf}(\mathbf{a}, L, k)$
6. **if** $\mathbf{a} = (1, \dots, 1)$
7. **return**

Visit All Leaves: Example




 Moving through all the leaves in order:



Depth First Search



- ❧ So we can search leaves
- ❧ How about searching all vertices of the tree?
- ❧ We can do this with a *depth first* search

3. Visit the Next Vertex



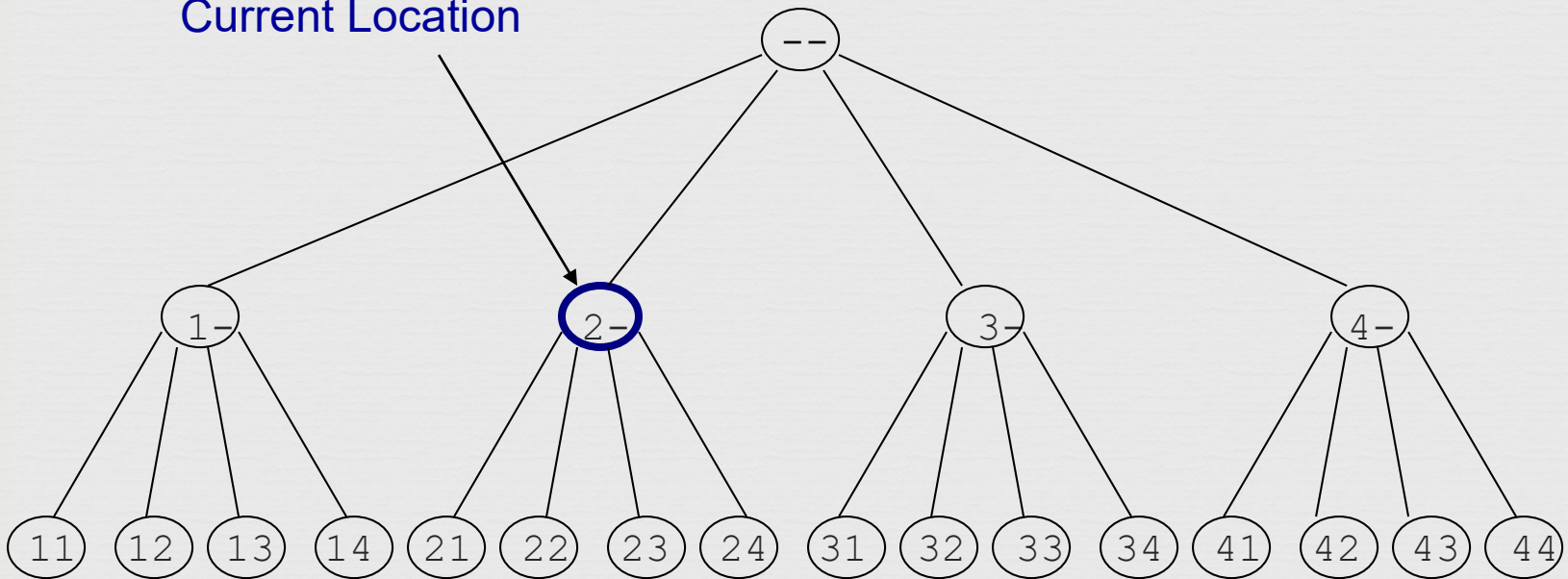
```
1. NextVertex(a,i,L,k)    // a : the array of digits
2.   if  $i < L$              // i : prefix length
3.      $a_{i+1} \leftarrow 1$     // L: max length
4.     return ( a,i+ 1)     // k: max digit value
5.   else
6.     for  $j \leftarrow 1$  to  $\ell$ 
7.       if  $a_j < k$ 
8.          $a_j \leftarrow a_j + 1$ 
9.         return( a,j)
10.  return(a,0)
```

Example



Moving to the next vertex:

Current Location

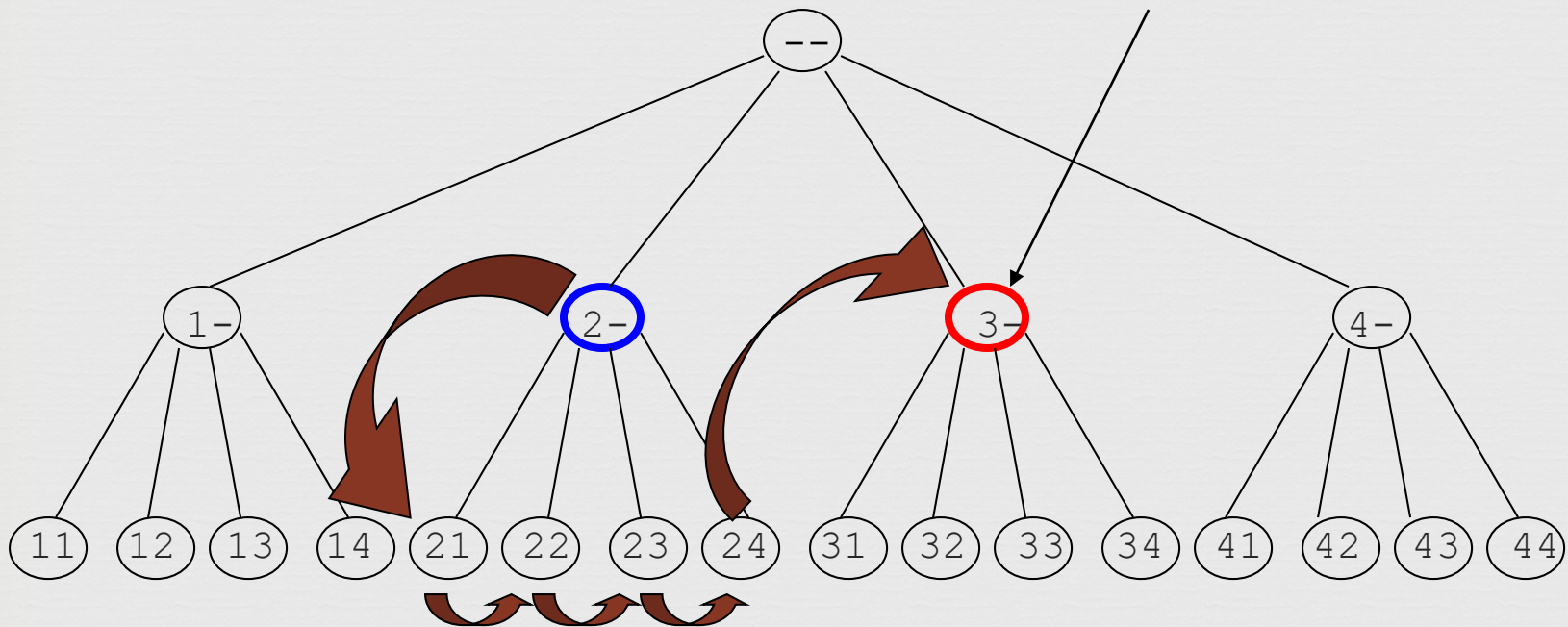


Example



Moving to the next vertices:

Location after 5
next vertex moves



4. Bypass the children



✧ Given a prefix (internal vertex), find next vertex after skipping all its children

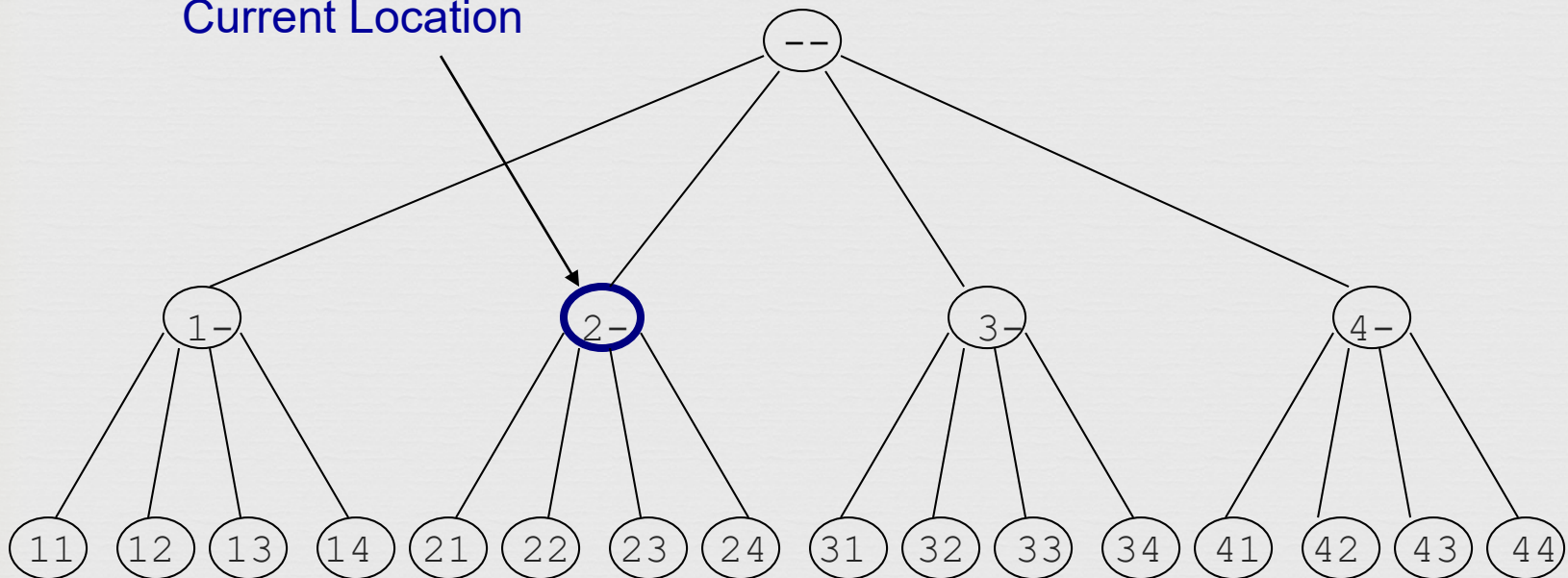
```
1.  Bypass(a,i,L,k)      // a: array of digits
2.  for  $j \leftarrow 1$  to  $i$     // i: prefix length
3.      if  $a_j < k$           // L: maximum length
4.           $a_j \leftarrow a_j + 1$  // k: max digit value
5.      return(a,j)
6.  return(a,0)
```

Bypass Move: Example




 Bypassing the descendants of “2-”:

Current Location

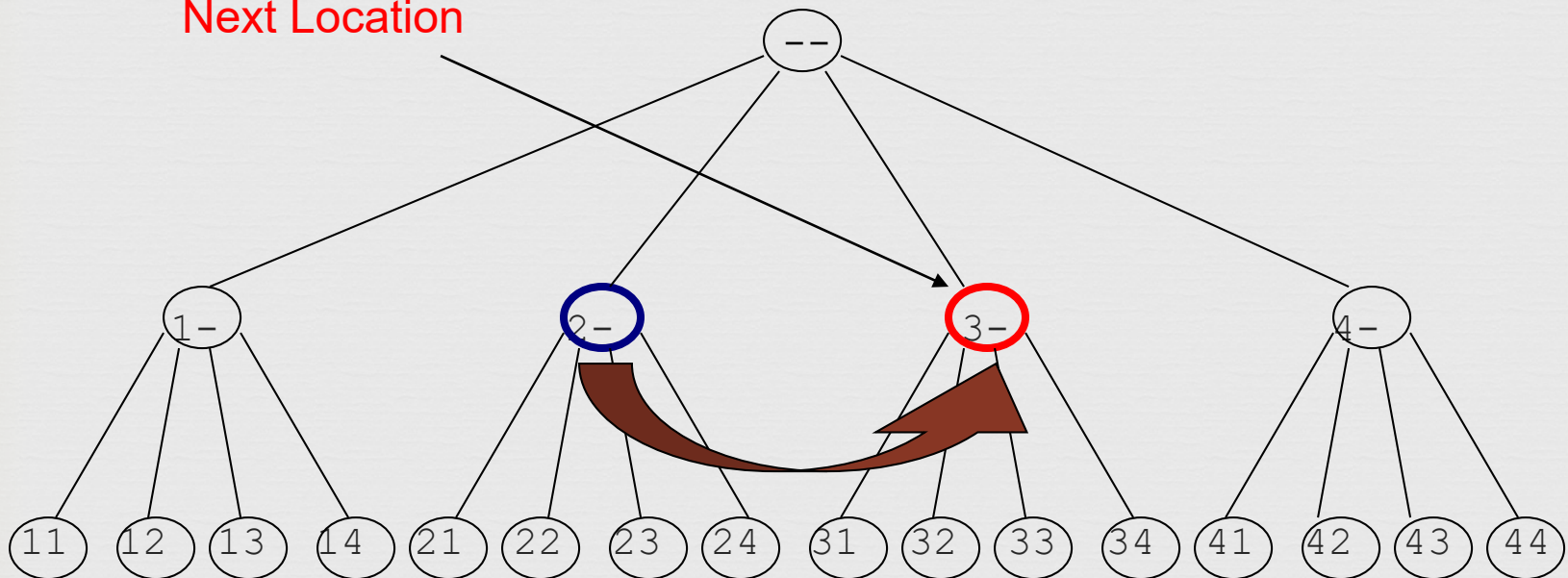


Example



By passing the descendants of “2-”:

Next Location



Outline



- ❧ What is Motif
- ❧ Motif finding problem
- ❧ Brute Force Motif Finding
- ❧ The Median String Problem
- ❧ Search Trees
- ❧ **Branch-and-Bound Motif Search**
- ❧ Branch-and-Bound Median String Search

Revisiting Brute Force Search



Now that we have method for navigating the tree,
lets look again at BruteForceMotifSearch

Brute Force Search Again



1. BruteForceMotifSearchAgain(*DNA*, *t*, *n*, *l*)
2. $s \leftarrow (1, 1, \dots, 1)$
3. $bestScore \leftarrow Score(s, DNA)$
4. while forever
5. $s \leftarrow \text{NextLeaf}(s, t, n - l + 1)$
6. if ($Score(s, DNA) > bestScore$)
7. $bestScore \leftarrow Score(s, DNA)$
8. $bestMotif \leftarrow (s_1, s_2, \dots, s_t)$
9. return $bestMotif$

HOW CAN WE IMPROVE THE SPEED?

Can We Do Better?



- ✧ Sets of $\mathbf{s}=(s_1, s_2, \dots, s_t)$ may have a weak profile for the first i positions $(s_1, s_2, \dots, s_i)$
- ✧ Every row of alignment may add at most ℓ to Score
- ✧ Optimism: if all subsequent $(t-i)$ positions $(s_{i+1}, \dots, s_t)$ add

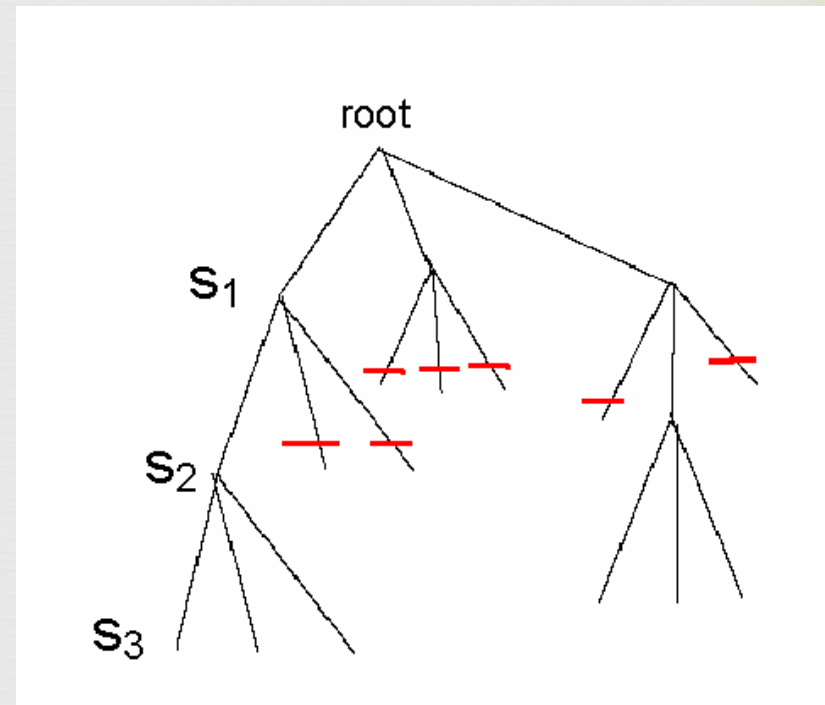
$$(t - i) * \ell \text{ to } \text{Score}(\mathbf{s}, i, \text{DNA})$$

- ✧ If $\text{Score}(\mathbf{s}, i, \text{DNA}) + (t - i) * \ell < \text{BestScore}$, it makes no sense to search in vertices of the current subtree
 - ✧ Use **ByPass()**

Branch and Bound Algorithm for Motif Search



-  Since each level of the tree goes deeper into search, discarding a prefix discards all following branches
-  This saves us from looking at $(n - \ell + 1)^{t-i}$ leaves
 -  Use **NextVertex()** and **ByPass()** to navigate the tree



Pseudocode for Branch and Bound Motif Search



```

1. BranchAndBoundMotifSearch(DNA, t, n, l)
2.  $s \leftarrow (1, \dots, 1)$ 
3.  $bestScore \leftarrow 0$ 
4.  $i \leftarrow 1$ 
5. while  $i > 0$ 
6.     if  $i < t$ 
7.          $optimisticScore \leftarrow Score(s, i, DNA) + (t - i) * l$ 
8.         if  $optimisticScore < bestScore$ 
9.              $(s, i) \leftarrow Bypass(s, i, n - l + 1)$ 
10.        else
11.             $(s, i) \leftarrow NextVertex(s, i, n - l + 1)$ 
12.        else
13.            if  $Score(s, DNA) > bestScore$ 
14.                 $bestScore \leftarrow Score(s)$ 
15.                 $bestMotif \leftarrow (s_1, s_2, s_3, \dots, s_t)$ 
16.             $(s, i) \leftarrow NextVertex(s, i, t, n - l + 1)$ 
17. return  $bestMotif$ 

```

Outline



- ❧ What is Motif
- ❧ Motif finding problem
- ❧ Brute Force Motif Finding
- ❧ The Median String Problem
- ❧ Search Trees
- ❧ Branch-and-Bound Motif Search
- ❧ **Branch-and-Bound Median String Search**

Median String Search Improvements



- Recall the computational differences between motif search and median string search
 - The Motif Finding Problem needs to examine all $(n-l+1)^t$ combinations for $\mathbf{S}$.
 - The Median String Problem needs to examine 4^l combinations of $\mathbf{v}$. This number is relatively small
- We want to use median string algorithm with the Branch and Bound trick!

Branch and Bound Applied to Median String Search



- ⌘ Note that if the total distance for a prefix is greater than that for the best word so far:

$$\text{TotalDistance}(\textit{prefix}, \textit{DNA}) > \textit{BestDistance}$$

there is no use exploring the remaining part of the word

- ⌘ We can eliminate that branch and **BYPASS** exploring that branch further

Bounded Median String Search



```

1.  BranchAndBoundMedianStringSearch(DNA, t, n, l)
2.   $s \leftarrow (1, \dots, 1)$ 
3.  bestDistance  $\leftarrow \infty$ 
4.   $i \leftarrow 1$ 
5.  while  $i > 0$ 
6.    if  $i < l$ 
7.      prefix  $\leftarrow$  string corresponding to the first  $i$  nucleotides of  $s$ 
8.      optimisticDistance  $\leftarrow$  TotalDistance(prefix, DNA)
9.      if optimisticDistance  $>$  bestDistance
10.         ( $s, i$ )  $\leftarrow$  Bypass( $s, i, l, 4$ )
11.      else
12.         ( $s, i$ )  $\leftarrow$  NextVertex( $s, i, l, 4$ )
13.    else
14.      word  $\leftarrow$  nucleotide string corresponding to  $s$ 
15.      if TotalDistance( $s, DNA$ )  $<$  bestDistance
16.         bestDistance  $\leftarrow$  TotalDistance(word, DNA)
17.         bestWord  $\leftarrow$  word
18.      ( $s, i$ )  $\leftarrow$  NextVertex( $s, i, l, 4$ )
19.  return bestWord

```

Speedup



HOW THE BRANCH AND BOUND TRICK
SPEEDUP THE ALGORITHMS?

More on the Motif Problem



- ✧ Exhaustive Search and Median String are both exact algorithms
- ✧ They always find the optimal solution, though they may be too slow to perform practical tasks
- ✧ Many algorithms sacrifice optimal solution for speed

References



- ❧ Phillip Compeau and Pavel Pevzner, *Bioinformatics Algorithms: An Active Learning Approach*, Chapter 2.
- ❧ Tognon, Manuel, Rosalba Giugno, and Luca Pinello. "A survey on algorithms to characterize transcription factor binding sites." *Briefings in Bioinformatics* 24, no. 3 (2023): bbad156.
- ❧ Santana, et. Al "Benchmarking tools for transcription factor prioritization." *Computational and Structural Biotechnology Journal* 23 (2024): 2190-2199.